

PHC DR Vision AI

Diabetic Retinopathy Screening System

Full Project Documentation

September 2026 · Version 1.0

■ CLINICAL DISCLAIMER

This is a research prototype and screening-support tool. It must not be used as a clinically validated diagnostic system. Final clinical assessment must be performed by a qualified ophthalmologist. A low-confidence result requires manual review and every referable result must be reviewed by an ophthalmologist according to the clinical workflow.

1. Project Overview

The PHC DR Vision AI is a Django web application designed for rural Primary Health Centres (PHC). A PHC worker uploads a fundus retinal image, enters patient demographic and clinical information, and the system returns an AI-assisted diabetic retinopathy (DR) classification along with a Grad-CAM heatmap. The system uses a pre-trained MATLAB ResNet-18 model (**trainedDRModel.mat**) without retraining or conversion.

1.1 Key Objectives

- Enable early DR detection at primary health centres with limited specialist access.
- Provide a five-class DR severity classification (No DR → Proliferative DR).
- Generate Grad-CAM heatmaps to visually explain model predictions.
- Implement employee access control: only admin-approved Employee IDs can register.
- Produce referral slips for referable DR cases (Moderate, Severe, Proliferative).

1.2 Technology Stack

Component	Technology
Backend Framework	Django 5.2
Database	MySQL 8.0 (phc_db)
AI Engine	MATLAB R2026a — ResNet-18 dlnetwork
Python–MATLAB Bridge	matlab.engine (MATLAB Engine for Python)
Image Processing	Pillow, NumPy
Frontend	HTML5, Bootstrap 5, Vanilla JavaScript
Authentication	Django Auth + Custom PHCWorker Backend
PDF Reports	ReportLab
Version Control	Git / GitHub

2. System Architecture

The system is a single Django project with three main layers: the web layer (Django views, templates, and REST-like JSON APIs), the AI engine layer (Python quality checks and MATLAB inference), and the persistence layer (MySQL and Django media storage).

2.1 Request Flow

- Browser submits fundus image + patient data via multipart POST to **/api/create_screening/**.
- Django saves the uploaded image to **media/fundus/**.
- Python quality checks validate sharpness, brightness, and field of view.
- If quality passes → MATLAB Engine runs **screen_fundus()** on the image.
- MATLAB returns: predicted class, DR level, confidence %, referable flag, Grad-CAM image.

- Django saves a **Screening** record to MySQL.
- Django returns a JSON response. Frontend renders results.

2.2 Project Structure

```
phc_screening/
├── manage.py
├── requirements.txt
├── .env.example
├── .gitignore
├── README.md
├── media/
│   ├── fundus/           # Uploaded fundus images
│   └── gradcam/          # Generated Grad-CAM overlays
├── phc_screening/
│   ├── settings.py       # Django settings (env-var driven)
│   ├── urls.py
│   ├── wsgi.py
│   └── asgi.py
├── screening/
│   ├── models.py        # Employee, PHCWorker, Patient, Screening, ReferralSlip
│   ├── views.py         # All views and API endpoints
│   ├── urls.py
│   ├── forms.py         # RegistrationForm, LoginForm, EmployeeForm
│   ├── backends.py      # Custom auth (username / email / phone)
│   ├── admin.py         # Django admin registration
│   ├── migrations/
│   └── templates/
│       ├── base.html
│       ├── base_no_nav.html
│       ├── dashboard.html
│       ├── admin_dashboard.html
│       ├── employee_form.html
│       ├── modals/
│       │   ├── new_screening_modal.html
│       │   └── referral_slip_modal.html
│       ├── registration/
│       │   ├── login.html
│       │   ├── register.html
│       │   └── admin_login.html
│       └── management/commands/
│           ├── setup_admin.py
│           ├── setup_demo_admin.py
│           └── setup_demo_doctor.py
├── ai_engine/
│   ├── matlab_engine.py  # MATLAB Engine singleton
│   ├── predictor.py      # Python inference bridge
│   ├── quality.py        # Image quality validation
│   └── matlab/
│       └── screen_fundus.m # MATLAB screening function
└── docs/
    └── PROJECT_DOCUMENTATION.md
```

3. Database Models

All models are defined in **screening/models.py** and backed by MySQL database **phc_db**.

3.1 Employee

Stores pre-approved Employee IDs created by the administrator. Only active, unregistered Employee IDs can be used at the /register/ page.

Field	Type	Description
employee_id	CharField(20, unique)	Pre-approved Employee ID
full_name	CharField(100)	Employee full name
role	CharField(80)	Job role (e.g. PHC Worker)
department	CharField(100)	PHC department / unit
phone	CharField(15, blank)	Contact phone
email	EmailField(blank)	Contact email
is_active	BooleanField	Controls registration access
user	OneToOneField(User, null)	Linked Django user after registration
created_at	DateTimeField(auto)	Record creation timestamp
updated_at	DateTimeField(auto)	Last update timestamp

3.2 PHCWorker

Extends the Django User model with PHC-specific fields, created automatically on registration.

Field	Type	Description
user	OneToOneField(User)	Linked Django user
phc_id	CharField(20, unique)	Employee ID used at registration
phone_number	CharField(15, unique)	Contact phone
phc_name	CharField(100)	PHC name
unit	CharField(100)	Screening unit

3.3 Patient

Field	Type	Description
first_name	CharField(50)	Patient first name
last_name	CharField(50, blank)	Patient last name
patient_id	CharField(20, unique)	Unique patient identifier
age	PositiveIntegerField	Patient age in years
gender	CharField(1)	M / F / O
diabetes_duration	DecimalField(4,1)	Years with diabetes (e.g. 1.5)
hba1c	DecimalField(4,1)	HbA1c value
created_at	DateTimeField(auto)	Record creation timestamp

3.4 Screening

Core model. One screening record is created per AI inference run.

Field	Type	Description
patient	ForeignKey(Patient)	Related patient
eye_side	CharField	LEFT / RIGHT
fundus_image	ImageField	Path to uploaded image
gradcam_image	ImageField(null)	Path to Grad-CAM overlay
predicted_class	CharField(30)	MATLAB class name
dr_level	IntegerField(default=0)	0 – 4 DR severity level
confidence	DecimalField(5,2)	Softmax confidence %
referable	BooleanField	True if DR level ≥ 2
low_confidence	BooleanField	True if confidence < 50%
quality_passed	BooleanField	Image quality gate result
quality_metrics	JSONField(null)	Sharpness, brightness, FOV
quality_message	TextField(blank)	Human-readable quality note
referral_urgency	CharField	routine / priority / urgent
referral_generated	BooleanField	Has referral slip been made?
screened_at	DateTimeField(auto)	Screening timestamp

4. AI Engine

4.1 Image Quality Checks (ai_engine/quality.py)

Before invoking MATLAB, every uploaded image passes through a Python quality gate:

Check	Threshold / Rule	Failure Message
Extension	JPG, JPEG, PNG only	Unsupported image format
File size	≤ 10 MB	File too large
Readability	Pillow can open image	Cannot read image
Sharpness	Laplacian variance ≥ 5	Image appears blurred
Brightness	Mean grey $\in [40, 220]$	Image too dark / overexposed
Field of view	Bright-pixel ratio $\geq 20\%$	Insufficient retinal coverage

If any check fails, Django returns HTTP **422** with the measured metrics. No patient or screening record is created and MATLAB is never called.

4.2 MATLAB Engine Lifecycle (ai_engine/matlab_engine.py)

A Python-level singleton (`_engine` + `_engine_lock`) ensures MATLAB is started only once per Django worker process. On the first screening request:

- `import matlab.engine` (raises `RuntimeError` if not installed)
- `matlab.engine.start_matlab()` starts an embedded MATLAB session.
- MATLAB path is extended with `ai_engine/matlab/` so `screen_fundus.m` is found.
- All subsequent requests in the same process reuse the live Engine.

Inside MATLAB, a **persistent** variable caches the loaded network, so trainedDRModel.mat is loaded only once per Engine session.

4.3 Inference Function (ai_engine/matlab/screen_fundus.m)

Django calls:

```
engine.screen_fundus(model_path, image_path, gradcam_output_path, nargout=6)
```

The MATLAB function executes these steps:

- Load net from trainedDRModel.mat (cached via persistent variable).
- imread(imageFile) — read the fundus image.
- Resize to net.Layers(1).InputSize using imresize.
- im2single → dldarray(input, 'SSC').
- predict(net, patientInput) → raw scores.
- softmax(extractdata(scores)) → class probabilities.
- max(probabilities) → predicted class index and confidence.
- Map index to classNames = [No_DR, Mild, Moderate, Severe, Proliferate_DR].
- DR level = class index – 1 (0 through 4).
- confidencePercent = confidence × 100.
- Grad-CAM computed using FeatureLayer='res5b_relu', ReductionLayer='prob'.
- Grad-CAM overlay saved as PNG to media/gradcam/.

4.4 Prediction Mapping (ai_engine/predictor.py)

MATLAB Class	DR Level	Referable	Referral Urgency
No_DR	0	No	Routine
Mild	1	No	Routine
Moderate	2	Yes	Priority (3–4 weeks)
Severe	3	Yes	Urgent (within 1 week)
Proliferate_DR	4	Yes	Urgent (within 1 week)

Low-confidence flag: **confidence < 50%** → low_confidence = True → triggers a clinical-review warning on the dashboard.

5. API Contract

5.1 Create Screening

POST /api/create_screening/

Content-Type: multipart/form-data

Auth: Required (login_required)

Form fields:

Field	Type	Required	Description
patientName	string	Yes	Patient full name
age	integer	Yes	Age in years
gender	string	Yes	M / F / O
diabetes_duration	decimal	Yes	Years with diabetes (e.g. 1.5)

Field	Type	Required	Description
hba1c	decimal	Yes	HbA1c value
eye_side	string	Yes	LEFT or RIGHT
patient_id	string	Yes	Unique patient ID from frontend
fundus_image	file	Yes	JPG/PNG fundus image ≤ 10 MB

Success response (HTTP 200):

```
{
  "id": 16,
  "patient_id": "PAT-20260908-001",
  "image_url": "/media/fundus/fundus_example.jpg",
  "gradcam_url": "/media/gradcam/fundus_example_gradcam.png",
  "predicted_class": "Moderate",
  "dr_level": 2,
  "confidence": 72.45,
  "referable": true,
  "referral_urgency": "priority",
  "referral_urgency_display": "Priority (3-4 Weeks)",
  "low_confidence": false
}
```

Response status codes:

Status	Meaning
200	Screening saved — MATLAB result returned
400	Bad request, MATLAB error, DB error, or missing fields
401/302	User not authenticated — redirect to /login/
405	HTTP method is not POST
422	Image failed quality validation

5.2 Other Endpoints

Method	URL	Auth	Description
GET	/api/patients/	Yes	List all patients for current user
GET	/api/screening/<patient_id>/	Yes	Get latest screening for a patient
GET	/api/referral/<screening_id>/	Yes	Generate referral slip data
GET	/	Yes	Dashboard (HTML)
GET	/register/	No	Registration page
POST	/register/	No	Submit registration form
GET	/login/	No	PHC worker login
GET	/admin-login/	No	Administrator login
GET	/admin-dashboard/	Staff	Employee management dashboard
POST	/admin-dashboard/employees/add/	Staff	Create new Employee ID
POST	/admin-dashboard/employees/<id>/edit/	Staff	Edit employee record

Method	URL	Auth	Description
POST	/admin-dashboard/employees/<id>/toggle-status/	Staff	Activate/deactivate

6. Authentication & Access Control

6.1 Custom Authentication Backend (screening/backends.py)

Users can log in with any of:

- Employee ID (PHC ID)
- Email address
- Phone number

The backend queries **PHCWorker.phc_id**, **User.email**, and **PHCWorker.phone_number** before falling back to Django's default username match.

6.2 Employee ID Access Control (screening/forms.py)

Registration at **/register/** validates the entered Employee ID against the **Employee** table. The form rejects registration if:

- Employee ID does not exist in the database.
- Employee record is `is_active = False` (deactivated by admin).
- Employee ID is already linked to a registered user.

Administrators create and manage Employee IDs at **/admin-dashboard/**. Deactivating an Employee record also deactivates the linked Django user account, immediately blocking login.

6.3 Default Admin Account

Field	Value
Admin Login URL	http://127.0.0.1:8000/admin-login/
Username	admin
Password	Admin@1234 (change after first login)
is_staff	True
is_superuser	True

The admin account is created/reset by running:

```
python manage.py setup_admin
```

7. Installation & Configuration

7.1 Prerequisites

- Python 3.11+
- MySQL 8.0+ with database `phc_db` created.
- MATLAB R2026a with Deep Learning Toolbox and Image Processing Toolbox.
- MATLAB Engine for Python installed in the same virtual environment.
- `trainedDRModel.mat` — ResNet-18 model file.

7.2 Installation Steps

1. Clone the repository

```

git clone https://github.com/reddy2045/DR-vision-AI.git
cd DR-vision-AI

# 2. Create virtual environment
python -m venv .venv
.\venv\Scripts\activate

# 3. Install Python dependencies
pip install -r requirements.txt

# 4. Install MATLAB Engine for Python
pip install "C:\Program Files\MATLAB\R2026a\extern\engines\python"

# 5. Verify MATLAB Engine
python -c "import matlab.engine; print('MATLAB Engine OK!)"

# 6. Copy environment template and fill in values
copy .env.example .env

```

7.3 Environment Variables

Variable	Default	Description
DJANGO_SECRET_KEY	insecure dev key	Django secret key — change for production
DJANGO_ALLOWED_HOSTS	localhost,127.0.0.1	Comma-separated allowed hosts
DB_NAME	phc_db	MySQL database name
DB_USER	root	MySQL username
DB_PASSWORD	(empty)	MySQL password
DB_HOST	localhost	MySQL server host
DB_PORT	3306	MySQL server port
MATLAB_MODEL_PATH	(required)	Full path to trainedDRModel.mat
EMAIL_HOST_USER	(empty)	Gmail/SMTP address for console fallback
EMAIL_HOST_PASSWORD	(empty)	Gmail app password

7.4 Database Setup

```

-- Create database in MySQL
CREATE DATABASE phc_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

-- Then run Django migrations
python manage.py migrate

-- Create default admin account
python manage.py setup_admin

-- Start development server (always use .venv Python)
.\venv\Scripts\python.exe manage.py runserver 0.0.0.0:8000

```

8. Error Handling & Troubleshooting

Error	Cause	Fix
MATLAB Engine not installed	matlab.engine ImportError on first request	Install from MATLAB R2026a extern/engines/python into .venv
ERR_CONNECTION_REFUSED	Django server not running	Run: .venv\Scripts\python.exe manage.py runserver
Table does not exist (1146)	Migration not applied	Run: python manage.py migrate
Invalid Employee ID	Employee ID not in database or inactive	Add/activate Employee ID in admin dashboard
Image quality 422	Fundus image failed quality gate	Recapture image — ensure good focus and lighting
Model not found	MATLAB_MODEL_PATH points to missing file	Set env var: \$env:MATLAB_MODEL_PATH='C:\path\to\model.mat'
Grad-CAM error	Layer names res5b_relu or prob not found in model architecture	Inspect model architecture in MATLAB; update screen_fundus.m layer names
Wrong Python interpreter	Django started with system Python, not venv	Always use .venv\Scripts\python.exe manage.py runserver

9. Security & Production Notes

9.1 Current Protections

- Login required on dashboard and all API endpoints.
- CSRF tokens enforced on all POST requests.
- File extension and size validation before processing.
- Pillow readability validation prevents malformed image uploads.
- No binary image data stored in MySQL — only file paths.
- No external AI API calls — inference runs locally.
- Database password and secret key read from environment variables.
- media/ and .env excluded from Git via .gitignore.

9.2 Required for Production

- Set DEBUG = False and configure ALLOWED_HOSTS.
- Use a strong random DJANGO_SECRET_KEY.
- Enable HTTPS with secure cookies (SESSION_COOKIE_SECURE, CSRF_COOKIE_SECURE).
- Serve media files via nginx or object storage — not Django dev server.
- Add rate limiting on the screening API.
- Implement antivirus / content scanning for uploaded images.
- Set up database backups.
- Move MATLAB inference to a background queue (Celery or dedicated service).
- Add audit logging for all screening and referral events.
- Implement formal clinical validation of quality thresholds and model accuracy.

10. Database Migrations History

Migration	Description
screening.0001_initial	Creates PHCWorker, Patient, Screening, ReferralSlip tables
screening.0002_phcworker_phone_number	Adds phone_number field to PHCWorker
screening.0003_screening_dr_level...	Adds dr_level, gradcam_image, confidence, referable, low_confidence, quality fields
screening.0004_alter_patient_diabetes_duration	Changes diabetes_duration to DecimalField (supports 1.5 years)

Migration	Description
screening.0005_employee	Creates Employee table; backfills from existing PHCWorker records

11. GitHub Repository

The source code is hosted at:

<https://github.com/reddy2045/DR-vision-AI>

What is included in the repository:

- All Python source files (views, models, forms, backends, predictor, quality).
- All Django templates (HTML).
- All database migrations.
- MATLAB helper function (screen_fundus.m).
- Project documentation (Markdown).
- .env.example — environment variable template.
- README.md — quick-start guide.
- .gitignore — excludes secrets, virtual envs, media, .mat files.

What is excluded from the repository:

- .venv/ and venv/ — install locally via requirements.txt.
- media/ — runtime-generated upload and Grad-CAM files.
- *.mat — MATLAB model file (must be provided separately).
- .env — local secrets (never commit passwords).

12. Clinical Disclaimer

The PHC DR Vision AI system is a **research prototype** intended for educational, demonstration, and screening-support purposes only.

■ *The model output is NOT a medical diagnosis.*

- The AI classification provides a confidence score and severity level as a screening support indicator only.
- A low-confidence result **REQUIRES** manual clinical review by a qualified ophthalmologist.
- Every result classified as Moderate, Severe, or Proliferative DR is marked as 'Referable' and must be reviewed by an ophthalmologist before any clinical decision is made.
- The system does not replace specialist diagnosis, clinical examination, or validated diagnostic instruments.
- No lesion counts (microaneurysms, hemorrhages, exudates) are generated. The model returns class probabilities only.
- Model accuracy has not been validated on the deployment population. Clinical validation must be performed before any patient-care use.